

Clase 4

JavaScript parte 2

IIC2513 - Tecnologías y Aplicaciones Web

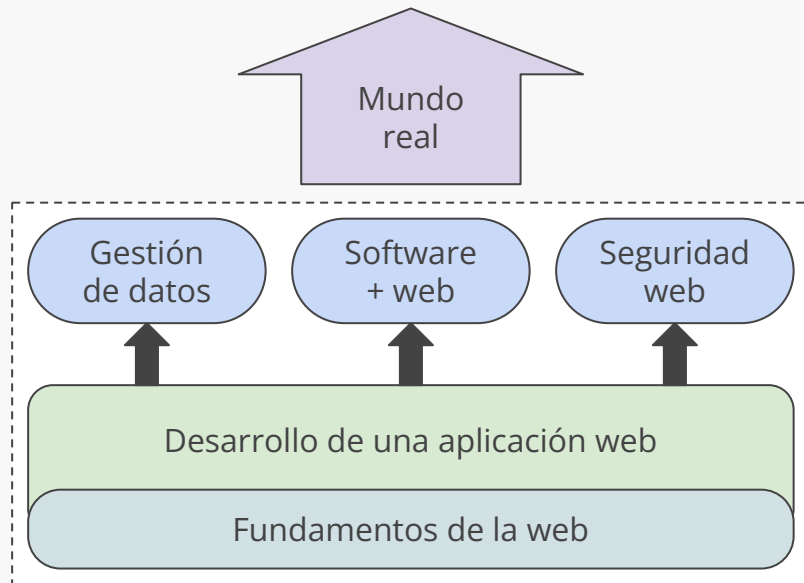
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Recuerden la ECA
2. Mañana hay ayudantía de JS

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

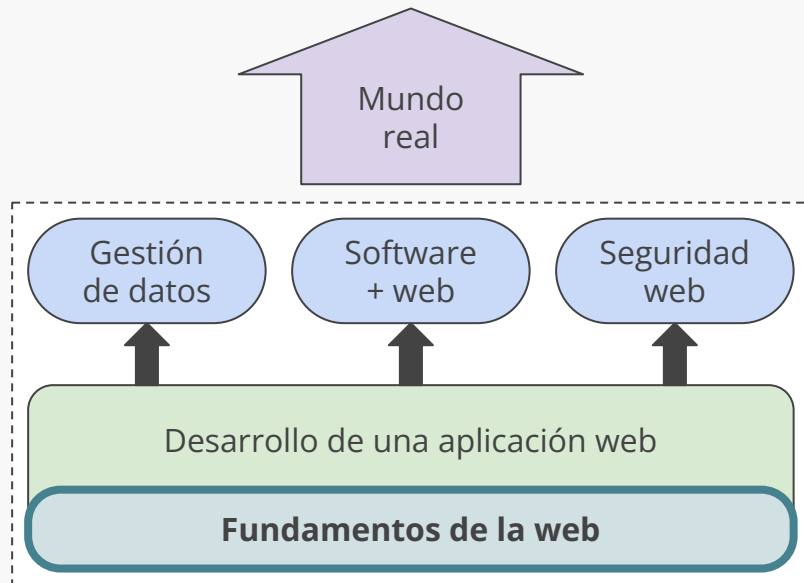


Contenidos del curso

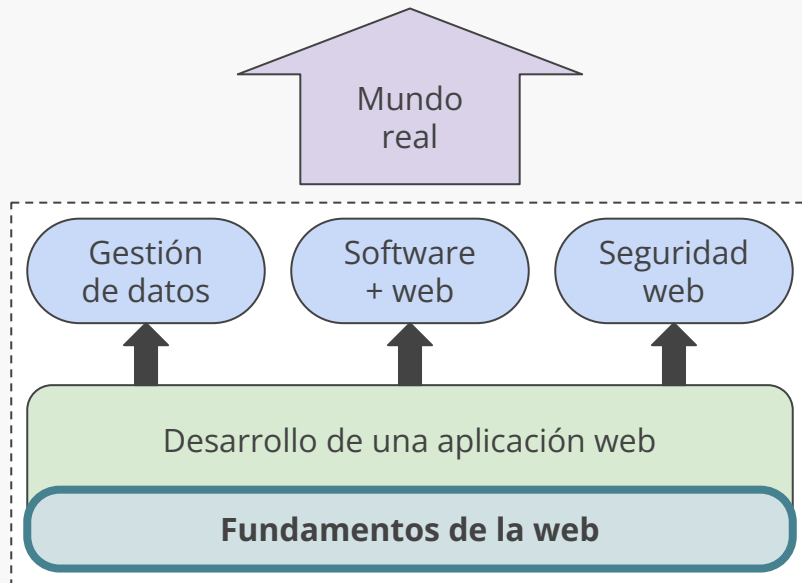
Fundamentos de la web

Veremos definiciones relacionadas con internet, protocolos y qué son HTML, CSS y **JavaScript...**

... para comprender la web como una **plataforma de desarrollo**, su funcionamiento básico, y protocolos y **tecnologías** detrás de esta



Contenidos del curso

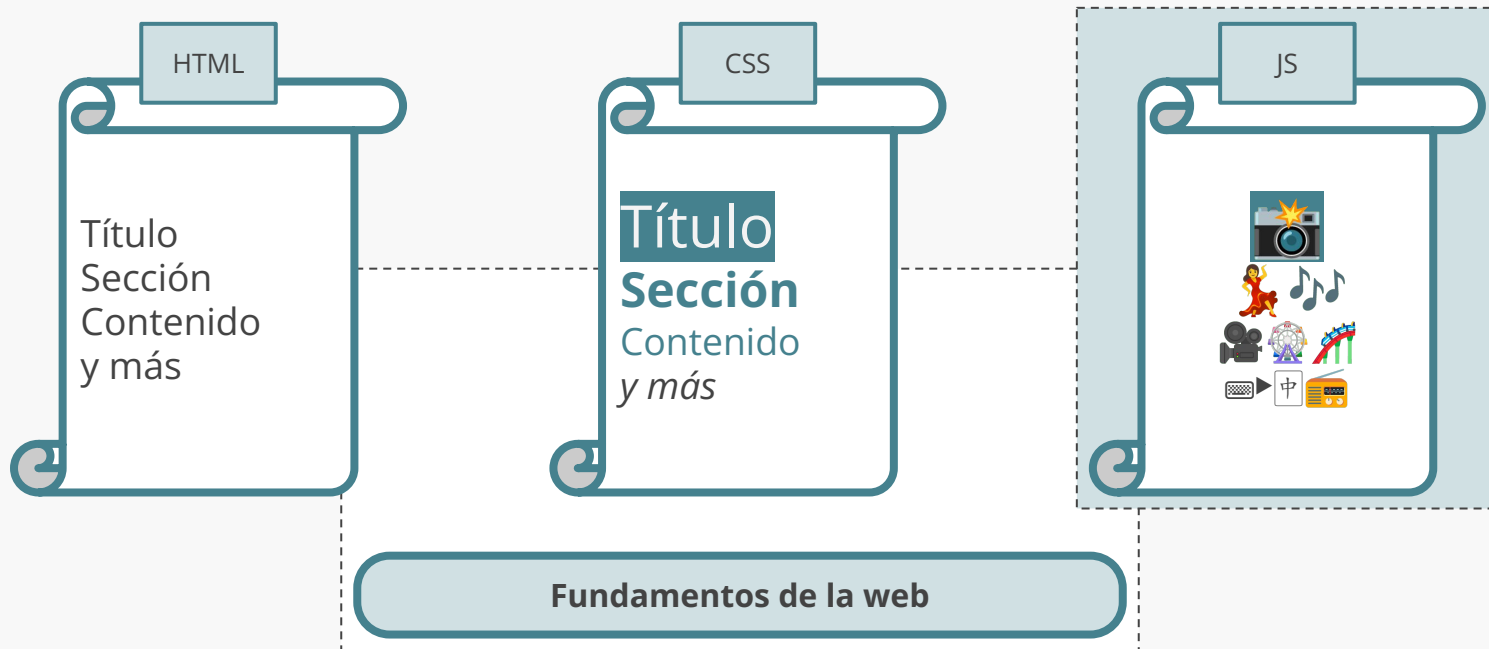


Contenidos del curso



Fundamentos de la web

Contenidos del curso

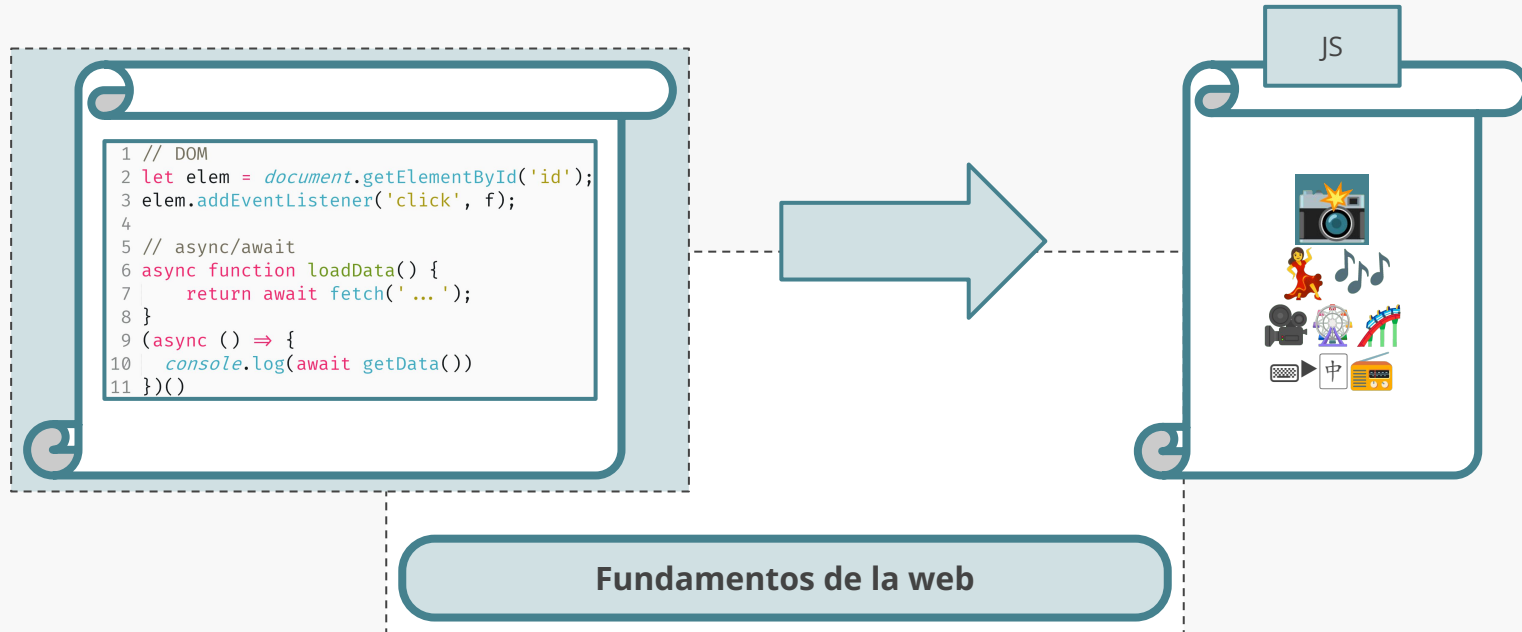


Contenidos del curso

Pregunta#¿Qué mecanismos de JavaScript son los más relevantes para el desarrollo web?

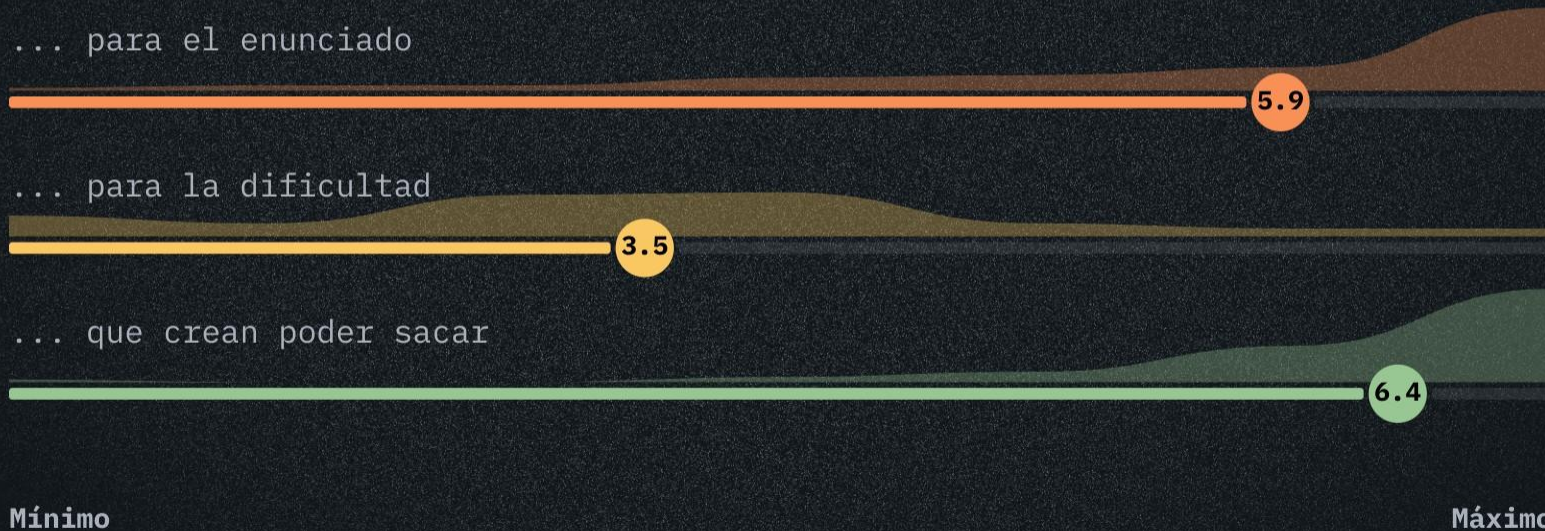


Contenidos del curso



Menti*#Volvamos a hablar de JavaScript*

Actividad 1 🧠 Pongan una nota...



¿Cuánto sabes de JavaScript (JS)? 🤖



¡Gracias por responder!

Agenda



Asincronía

Manejo de DOM

Código síncrono y asíncrono

Un **código síncrono** es aquel código donde cada instrucción espera a la anterior para ejecutarse mientras que un **código asíncrono** no espera a las instrucciones diferidas y continúa con su ejecución



Ejemplo síncrono



Ejemplo asíncrono

Ejemplo: Código síncrono

El código síncrono ejecuta cada línea de código, línea por línea, según el orden en que se encuentran escritas

La salida de este ejemplo es:

Inicio
Procesamiento...
Final

```
1 console.log('Inicio');  
2  
3 console.log('Procesamiento ... ');  
4  
5 console.log('Final');
```


Ejemplo: Versión asíncrona

Por otra parte, en el caso del código asíncrono no se detiene la ejecución y no se espera a que una línea acabe

La salida de este ejemplo es:

Inicio

Final

(y 2 segundos después...)

Procesamiento...

```
1 console.log('Inicio');  
2  
3 setTimeout(() => {  
4     console.log('Procesamiento');  
5 }, 2000);  
6  
7 console.log('Final');
```

¿Qué tiene que ver con la web?

En el código usado en las aplicaciones Web es muy común que la ejecución deba esperar que suceda algo para seguir adelante:

- Cuando una solicitud es respondida por un servidor
- Cuando el usuario presiona una tecla
- Cuando el usuario hace clic en algo
- ... y prácticamente en todas las acciones/interacciones del usuario en el navegador

La acción a seguir al responder a un evento por lo general implica hacer cambios en el documento que está desplegado en pantalla (es decir, manejo del DOM)

La forma más simple de manejarlo es mediante lo que se denomina **un *callback***

Agenda

Asincronía

✨ *Callbacks*

Promesas

Funciones *async*

Manejo de DOM

Callbacks

Una función de *callback* es una función que se pasa a otra función como un argumento, que luego se invoca dentro de la función externa para completar algún tipo de rutina o acción

```
function primeraFuncion() {  
    // Hacer algo  
}  
  
function otraFuncion(callback) {  
    // Hacer algo  
    callback(); // Llamada al callback  
}  
  
otraFuncion(primerFuncion);
```

Ejemplo de un *callback*

```
readFile("example.txt", function(err, contents) {  
  if (err) {  
    throw err;  
  }  
  console.log(contents);  
});  
console.log("Hi!");
```

- `readFile()` ejecuta de inmediato y pausa mientras se lee el archivo
- "Hi!" aparece de inmediato
- Función es ejecutada cuando se completa la lectura del archivo
- Finalmente se despliega el contenido

Tipos de *callbacks*

Callbacks síncronos donde la ejecución es inmediata (como map)

```
array.map(elem => console.log(elem));
```

Callbacks asíncronos donde la ejecución se pospone (como setTimeout)

```
setTimeout(function () {  
    console.log('Waited 3 seconds');  
}, 3000);
```

Estructura general de un *callback*

Aquí se usa
el *callback*

```
function funcionAsincrona(unCallback) {  
  // Operación asíncrona como leer un archivo  
  // Simplemente no sabemos cuánto va a tardar  
  // Y es algo que no queremos detenga la ejecución  
  // ...  
  const data = obtenerDatosDesdeAlgunaParte();  
  unCallback(data);  
}
```

```
// Primera alternativa, pasar una función ya definida  
function esteEsUnCallback(param) {  
  console.log(param);  
}  
funcionAsincrona(esteEsUnCallback);
```

Esta función es
un *callback*

```
// Otra alternativa es usar una función expresión  
funcionAsincrona(function (param) {  
  console.log(param);  
});
```

Esta también

```
// Otra alternativa es usar una función flecha  
funcionAsincrona(param => console.log(param));
```

Y esta

Un *callback* puede llamar otra función asíncrona

Dentro del *callback* entregado en la línea 3, llamamos a otro *callback* en la línea 5 y así...

```
let iniciarProceso = function () {  
  console.log('Iniciando proceso');  
  setTimeout(function () {  
    console.log('Llamando un callback');  
    setTimeout(function () {  
      console.log('Segundo callback...');  
      setTimeout(function () {  
        console.log('Otro callback???');  
      }, 1000);  
    }, 1000);  
  }, 1000);  
};  
  
iniciarProceso();
```


Callback hell: anidación múltiple

Haciendo múltiples veces lo anterior...
podríamos llegar a algo así

```
let cocinar = function () {  
  setTimeout(function () {  
    console.log('Empiezo a cocinar');  
    setTimeout(function () {  
      console.log('Seleccionar ingredientes');  
      setTimeout(function () {  
        console.log('Paso 1... ');  
        setTimeout(function () {  
          console.log('Paso 2...');  
          setTimeout(function () {  
            console.log('Paso 3...');  
            // ...  
          }, 1000);  
        }, 1000);  
      }, 3000);  
    }, 2000);  
  }, 1000);  
};  
cocinar();
```

Callback hell: ejemplo real

```
fs.readdir(source, function (err, files) {
  if (err) {
    console.log('Error finding files: ' + err)
  } else {
    files.forEach(function (filename, fileIndex) {
      console.log(filename)
      gm(source + filename).size(function (err, values) {
        if (err) {
          console.log('Error identifying file size: ' + err)
        } else {
          console.log(filename + ' : ' + values)
          aspect = (values.width / values.height)
          widths.forEach(function (width, widthIndex) {
            height = Math.round(width / aspect)
            console.log('resizing ' + filename + 'to ' + height + 'x' + height)
            this.resize(width, height).write(dest + 'w' + width + '_' + filename, function(err) {
              if (err) console.log('Error writing file: ' + err)
            })
          })
          this.bind(this))
        }
      })
    })
  }
})
```

Callback Hell: Guía sobre cómo escribir programas asíncronos (sin *callback hell*)

Callback Hell

A guide to writing asynchronous JavaScript programs

What is "callback hell"?

Asynchronous JavaScript, or JavaScript that uses callbacks, is hard to get right intuitively. A lot of code ends up looking like this:

```
fs.readdir(source, function (err, files) {
  if (err) {
    console.log('Error finding files: ' + err)
  } else {
    files.forEach(function (filename, fileIndex) {
      console.log(filename)
      gm(source + filename).size(function (err, values) {
        if (err) {
          console.log('Error identifying file size: ' + err)
        } else {
          console.log(filename + ' : ' + values)
          aspect = (values.width / values.height)
          widths.forEach(function (width, widthIndex) {
            height = Math.round(width / aspect)
            console.log('resizing ' + filename + 'to ' + height + 'x' + height)
            this.resize(width, height).write(dest + 'w' + width + '_' + filename, function(err) {
              if (err) console.log('Error writing file: ' + err)
            })
          })
        }
      }).bind(this))
    })
  }
})
}
```

Agenda

Asincronía

Callbacks



Promesas

Funciones *async*

Manejo de DOM

Promesas (*Promises*)

Objetos que representan el **estado intermedio** de una operación cuyo valor será resuelto en el **futuro**, es decir:

Es la promesa de un valor futuro

*"No puedo retornar un valor ahora, pero **prometo** que retornaré un valor en algún punto"*

- Una promesa en JavaScript (2025)

```
>> new Promise(() => {});  
← Promise { <state>: "pending" }  
  |  
  <state>: "pending"  
  ▶ <prototype>: Promise.prototype { ... }
```

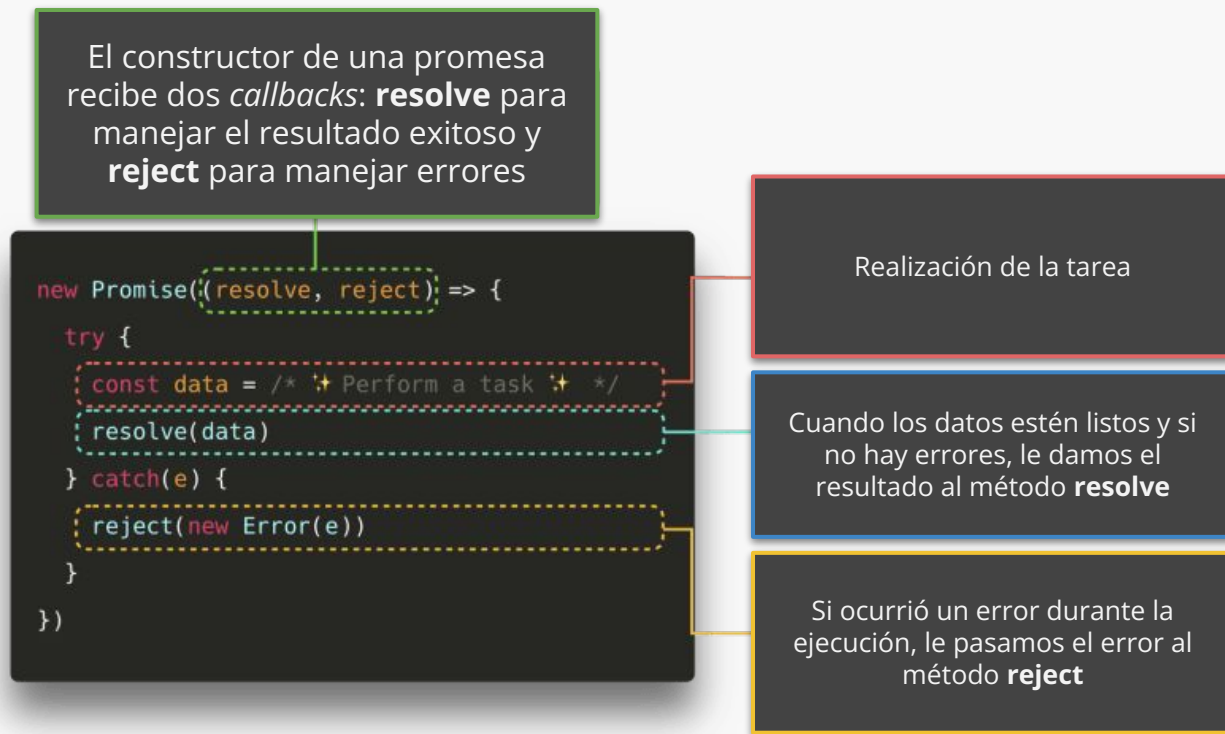
Promesas (*Promises*)

- Simplifica el trabajo especialmente cuando hay callbacks anidadas
- Una promesa representa el término eventual (exitoso o fallido) de una operación asíncronica y su resultado

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve('Success'), 1000);  
});  
promise.then(result => console.log(result));
```

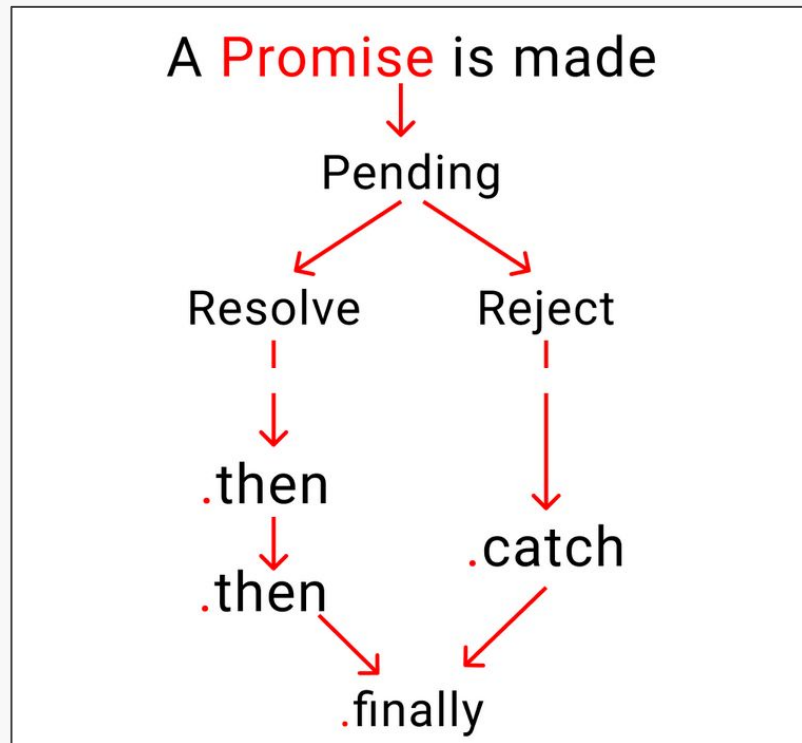
- La mayoría de las veces uno no crea promesas sino que recibe una como resultado de una operación de alguna librería o función

Constructor de una promesa



Estados de una promesa

- **Pending**
Estado inicial en el que aún no ocurre nada
- **Resolved**
La promesa retornó un valor sin errores
- **Rejected**
La promesa no se pudo completar



JavaScript Async/Await Tutorial (by Joy Shaheb @ freeCodeCamp):

Lectura recomendada para repasar y entender algunos de los conceptos que veremos durante esta clase

Search 7,400+ tutorials

freeCodeCamp (🔥)

Forum

Donate

JUNE 2, 2021 / #ASYNC/AWAIT

JavaScript Async/Await Tutorial – Learn Callbacks, Promises, and Async/Await in JS by Making Ice Cream 🍦🍧🍦



Joy Shaheb

Learn Async / await Promises

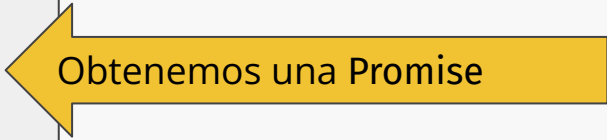


¿Y cómo obtengo el valor prometido?

La función `miFuncionAsincrona` es una función que **retorna una promesa**

```
function miFuncionAsincrona() {  
  return new Promise((resolve, reject) => {  
    resolve('Prometí este valor');  
  });  
}
```

```
const valorPrometido = miFuncionAsincrona();  
console.log(valorPrometido); // Una promesa
```



Obtenemos una Promise

¿Y cómo obtengo el valor prometido?

- Una promesa admite un método `then` que como parámetro recibe la función a llamar cuando la promesa se hace efectiva (un *callback*)
- El método `then` devuelve una promesa lo que hace posible encadenar una promesa tras otra

```
function miFuncionAsincrona() {  
  return new Promise((resolve, reject) => {  
    resolve('Prometí este valor');  
  });  
}  
  
miFuncionAsincrona().then(function (valorPrometido) {  
  console.log(valorPrometido);  
});  
  
miFuncionAsincrona()  
  .then(function (valorPrometido) {  
    return `${valorPrometido} es el valor prometido`;  
  })  
  .then(function (valorPrometido) {  
    console.log(valorPrometido);  
  });
```

Manejo de errores (promesas rechazadas)

Si se usa **reject** o ocurre una excepción en una promesa, esta se considera rechazada. En este caso, podemos ejecutar *callback* al usar **catch** en lugar de **then**:

```
new Promise((resolve, reject) => {
  resolve('Valor resuelto');
})
.then(valor => {
  throw new Error(`No me gusta este valor: ${valor}`);
  return 'Solo pasa si no hay error';
})
.catch(msg => {
  console.log(`Se rechazó con: ${msg}`);
  Se rechazó con: Error: No me gusta este valor: Valor resuelto
  return 'Estamos en una promesa, hay que resolver';
})
.then(otroValorHuh => {
  console.log(`Lo que viene de la promesa previa: ${otroValorHuh}`);
  Lo que viene de la promesa previa: Estamos en una promesa, hay que resolver
});
```

Manejo de errores (promesas rechazadas)

Si se usa **reject** o ocurre una excepción en una promesa, esta se considera rechazada. En este caso, podemos ejecutar *callback* al usar **catch** en lugar de **then**:

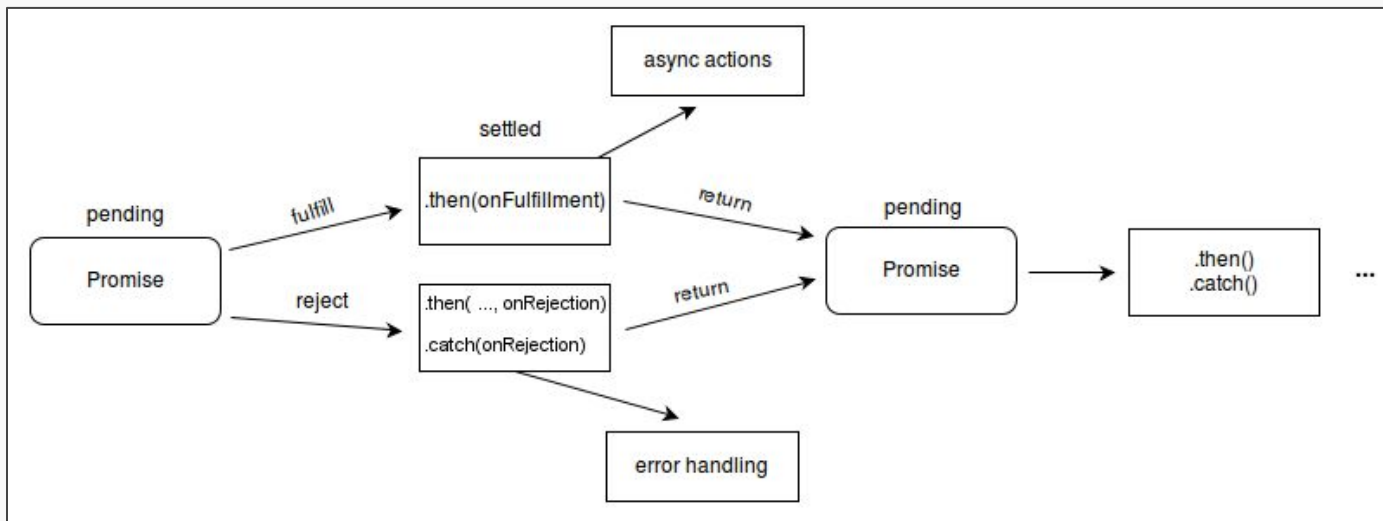
```
new Promise((resolve, reject) => {
  resolve('Valor resuelto');
})
.then(valor => {
  // throw new Error(`No me gusta este valor: ${valor}`);
  return 'Solo pasa si no hay error';
})
.catch(msg => {
  console.log(`Se rechazó con: ${msg}`);
  return 'Estamos en una promesa, hay que resolver';
})
.then(otroValorHuh => {
  console.log(`Lo que viene de la promesa previa: ${otroValorHuh}`);
  Lo que viene de la promesa previa: Solo pasa si no hay error
});
```

Manejo de errores (promesas rechazadas)

Si se usa **reject** o ocurre una excepción en una promesa, esta se considera rechazada. En este caso, podemos ejecutar *callback* al usar **catch** en lugar de **then**:

```
new Promise((resolve, reject) => {
  reject('Valor rechazado');
})
  .then(valor => {
    throw new Error(`No me gusta este valor: ${valor}`);
    return 'Solo pasa si no hay error';
  })
  .catch(msg => {
    console.log(`Se rechazó con: ${msg}`);
    Se rechazó con: Valor rechazado
    return 'Estamos en una promesa, hay que resolver';
  })
  .then(otroValorHuh => {
    console.log(`Lo que viene de la promesa previa: ${otroValorHuh}`);
    Lo que viene de la promesa previa: Estamos en una promesa, hay que resolver
  });
```

Resumen de estados y métodos de promesas



¿Y el *callback hell*?

```
// Cadena de callbacks
seleccionarReceta((ingredientes) => {
  seleccionarIngredientes(ingredientes, (ingredientesSeleccionados) => {
    let ingredientesCortados = [];
    let ingredientesPorCortar = ingredientesSeleccionados.length;

    ingredientesSeleccionados.forEach((ingrediente) => {
      cortarIngrediente(ingrediente, (ingredienteCortado) => {
        ingredientesCortados.push(ingredienteCortado);
        ingredientesPorCortar--;

        if (ingredientesPorCortar === 0) {
          console.log('Todos los ingredientes están cortados:', ingredientesCortados);
        }
      });
    });
  });
  recetaPaso1(() => {
    recetaPaso2(() => {
      recetaPaso3(() => {
        console.log('Receta completada');
      });
    });
  });
});
});
```


¿Y el *callback hell*? Arreglado con *Promises*

```
// Cadena de promesas
seleccionarReceta()
  .then(seleccionarIngredientes)
  .then(ingredientes => Promise.all(ingredientes.map(cortarIngrediente)))
  .then(ingredientesCortados => {
    return recetaPaso1(ingredientesCortados)
      .then(() => recetaPaso2(ingredientesCortados))
      .then(() => recetaPaso3(ingredientesCortados));
  })
  .then(() => console.log('Receta completada'));
```

Un ejemplo: then, catch y finally

```
let momsPromise = new Promise(function(resolve, reject) {
  let momsSavings = 1000000;
  let priceOfPhone = 800000;
  if (momsSavings > priceOfPhone) {
    resolve({ brand: "iphone", model: "14" });
  } else {
    reject("We dont have enough savings. Let us save some more money.");
  }
});
momsPromise.then(function(value) {
  console.log("Hurray I got this phone as a gift ", JSON.stringify(value));
});
momsPromise.catch(function(reason) {
  console.log("Mom couldn't buy me the phone because ", reason);
});
momsPromise.finally(function() {
  console.log("Irrespective of whether my mom can buy me a phone or not, I still love her");
});
```

Un ejemplo: ¿cuáles son los posibles *outputs*?

```
let cleanRoom = function() {
  return new Promise(function(resolve, reject) {
    resolve('cleaned the room');
  });
};

let removeGarbage = function(message) {
  return new Promise(function(resolve, reject) {
    resolve(message + ' remove garbage');
  });
};

let winIcecream = function(message) {
  return new Promise(function(resolve, reject) {
    resolve(message + ' won icecream');
  });
};

cleanRoom()
  .then(function(result) {
    return removeGarbage(result);
  })
  .then(function(result) {
    return winIcecream(result);
  })
  .then(function(result) {
    console.log('finished ' + result);
  });
```

Un ejemplo web: haciendo *fetch* de un JSON

```
function fetchData(url) {  
  return fetch(url)  
    .then(response => response.json())  
    .then(data => console.log(data))  
    .catch(err => console.error(`Hubo un error: ${err}`));  
}  
  
const url = 'https://jsonplaceholder.typicode.com/users';  
fetchData(url);
```

Agenda

Asincronía

Callbacks

Promesas

✨ **Funciones *async***

Manejo de DOM

async y await

Introducidos en ES2017, *async* y *await* son *syntactic sugar* sobre promesas para escribir código de forma que parezca código síncrono...



async functions

La *keyword* **async** se antepone a una función para que automáticamente retorne una promesa sin necesitar el constructor

No se usa **reject** ni **resolve**, sino que estos casos se manejan externamente con **then** y **catch**, respectivamente

Importante: Sólo funciona dentro de una *async function*

```
async function functionQuePromete() {  
  return 'Han pasado 84 años';  
}
```

```
let valor = functionQuePromete();  
console.log(valor);
```

```
Promise { 'Han pasado 84 años' }
```

await keyword

La *keyword* **await** se antepone a una promesa para esperar su resultado, emulando comportamiento síncrono

(Recordar que **funcionAEsperar** es una promesa porque usa **async**)

Importante: Sólo funciona dentro de una *async function*

```
async function functionQuePromete() {  
  return 'Han pasado 84 años';  
}  
  
async function functionQueEspera() {  
  const value = await functionQuePromete();  
  console.log(`Esperaba por ${value}`);  
  return 'Ahora yo retorno';  
}  
  
functionQueEspera();
```

Esperaba por Han pasado 84 años

Ejemplo de promesas vs. *await*

```
fetch('coffee.jpg')
  .then(response => response.blob())
  .then(myBlob => {
    let img = document.createElement('img');
    img.src = URL.createObjectURL(myBlob);
    document.body.appendChild(img);
  })
  .catch(e => {
    console.log(`Error ${e.message}`);
  });
```

```
async function myFetch() {
  let response = await fetch('coffee.jpg');
  let myBlob = await response.blob();

  let img = document.createElement('img');
  img.src = URL.createObjectURL(myBlob);
  document.body.appendChild(img);
}

myFetch()
  .catch(e => {
    console.log(`Error ${e.message}`);
  });
```

Agenda

Asincronía

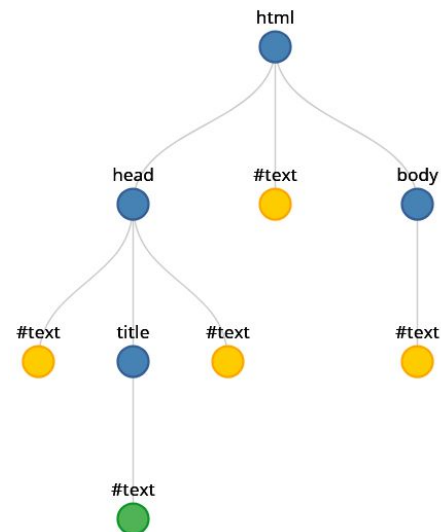
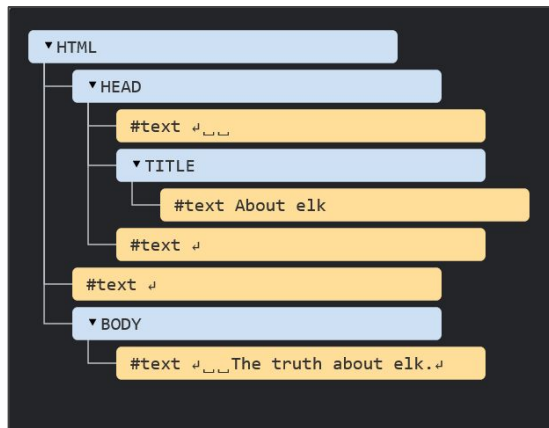


Manejo de DOM

HTML DOM

La representación interna de un documento HTML se llama **DOM (document object model)**. El DOM es un modelo jerárquico, basado en un árbol, donde *tags* y todo elemento en el HTML son representados como nodos, construyéndose según la jerarquía de la página desde el *tag* html. Estandarizado por la W3C ([ver estándar](#))

```
<!DOCTYPE HTML>
<html>
<head>
  <title>About elk</title>
</head>
<body>
  The truth about elk.
</body>
</html>
```



HTML DOM

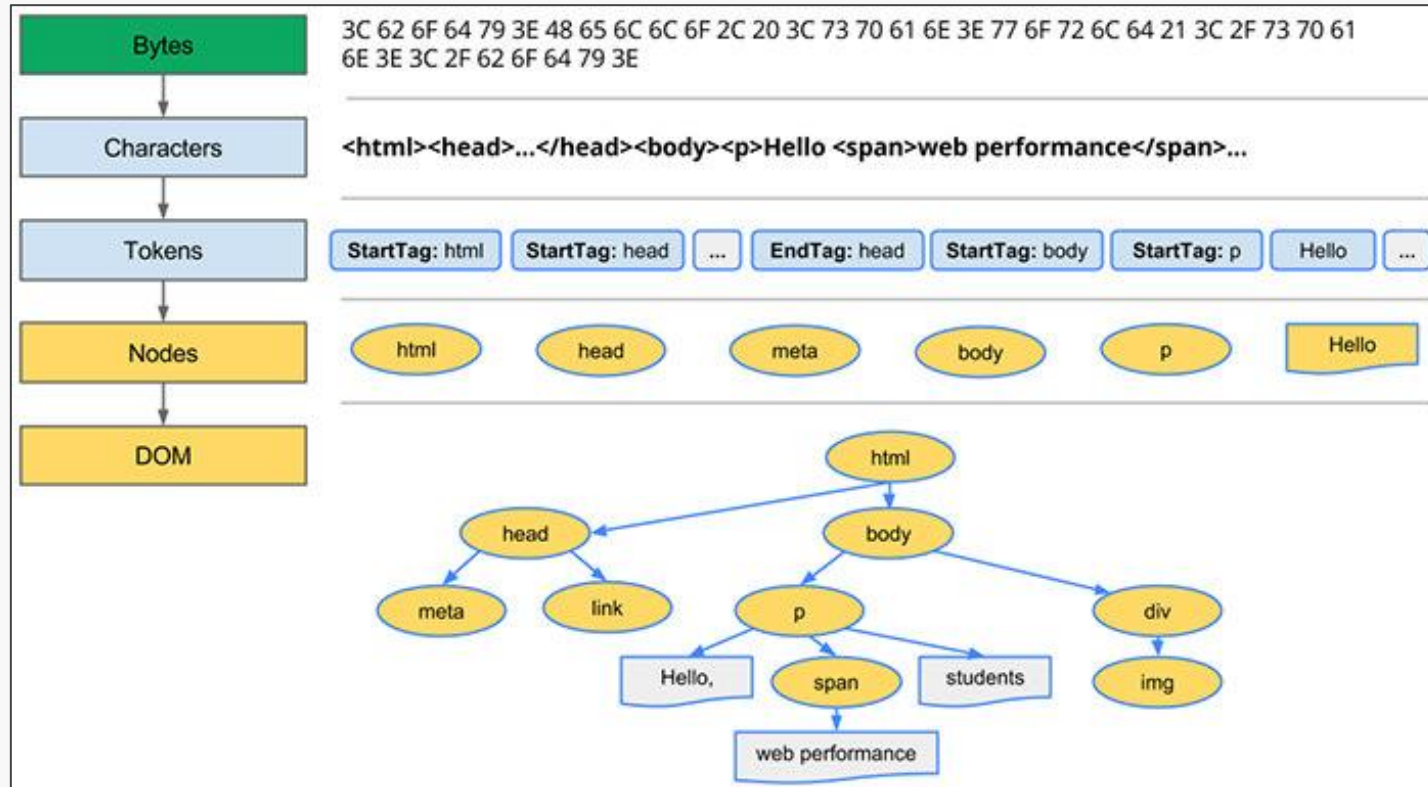


Figura: [The Beginners Guide to CSS Object Model \(CSSOM\)](#) | Hongkiat

HTML DOM desde JavaScript

El HTML DOM es un modelo de objeto estandarizado y una interfaz de programación para HTML. Define:

- Los elementos HTML como **objetos**
- Las **propiedades** de todos los elementos HTML
- Los **métodos** para acceder a todos los elementos HTML
- Los **eventos** para todos los elementos HTML

Esto convierte al HTML DOM es un estándar sobre cómo obtener, cambiar, agregar o eliminar elementos HTML:

- **JavaScript** puede hacer consultas y cambios sobre el DOM
- Accesible como **variable global** en el navegador (window, this)

Agenda

Asincronía

Manejo de DOM

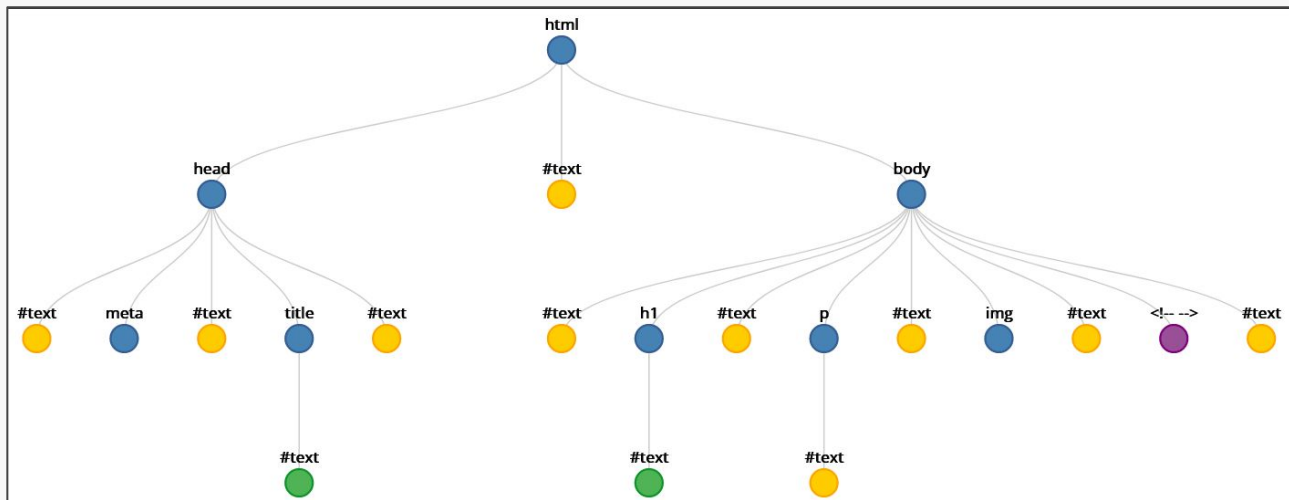


Manejo de elementos

Respondiendo a eventos

Jerarquía de nodos

El nodo raíz (tag `<html>`) se encuentra en `window.document` y sigue la estructura HTML:
`window.document.head` y `window.document.body`



Búsqueda de nodos: por id

Si un elemento tiene valor de id, podemos recuperar ese elemento usando el método `document.getElementById(id)`, sin importar donde esté

```
1 <div id="elem">
2   <div id="elem-content">Element</div>
3 </div>
4
5 <script>
6   // get the element
7   let elem = document.getElementById('elem');
8
9   // make its background red
10  elem.style.background = 'red';
11 </script>
```


Búsqueda de nodos: por selector CSS

Es un método versátil, ya que `elem.querySelectorAll(css)` retorna todos los elementos dentro del elemento `elem` que hacen *match* con el selector CSS entregado. En este ejemplo, buscamos todos los `<li>` que son el último elemento de su *parent*:

```
1 <ul>
2   <li>The</li>
3   <li>test</li>
4 </ul>
5 <ul>
6   <li>has</li>
7   <li>passed</li>
8 </ul>
9 <script>
10   let elements = document.querySelectorAll('ul > li:last-child');
11
12   for (let elem of elements) {
13     alert(elem.innerHTML); // "test", "passed"
14   }
15 </script>
```

Búsqueda de nodos: `getElementsBy*`

Además, existen otros métodos usados (aunque el más común y poderoso hoy en día es `querySelector`):

- `elem.getElementsByTagName(tag)` busca elementos con el *tag* entregado y retorna una colección de elementos
- `elem.getElementsByClassName(className)` entrega los elementos que tengan la clase CSS entregada
- `document.getElementsByName(name)` entrega los elementos que tengan dicho valor en el atributo `name`

Búsqueda de nodos

Método	Busca por...	¿Puede llamar a un elemento?	¿Vivo?
<code>querySelector</code>	selector CSS	✓	-
<code>querySelectorAll</code>	selector CSS	✓	-
<code>getElementById</code>	<code>id</code>	-	-
<code>getElementsByName</code>	name	-	✓
<code>getElementsByTagName</code>	etiqueta o <code>'*'</code>	✓	✓
<code>getElementsByClassName</code>	class	✓	✓

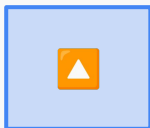
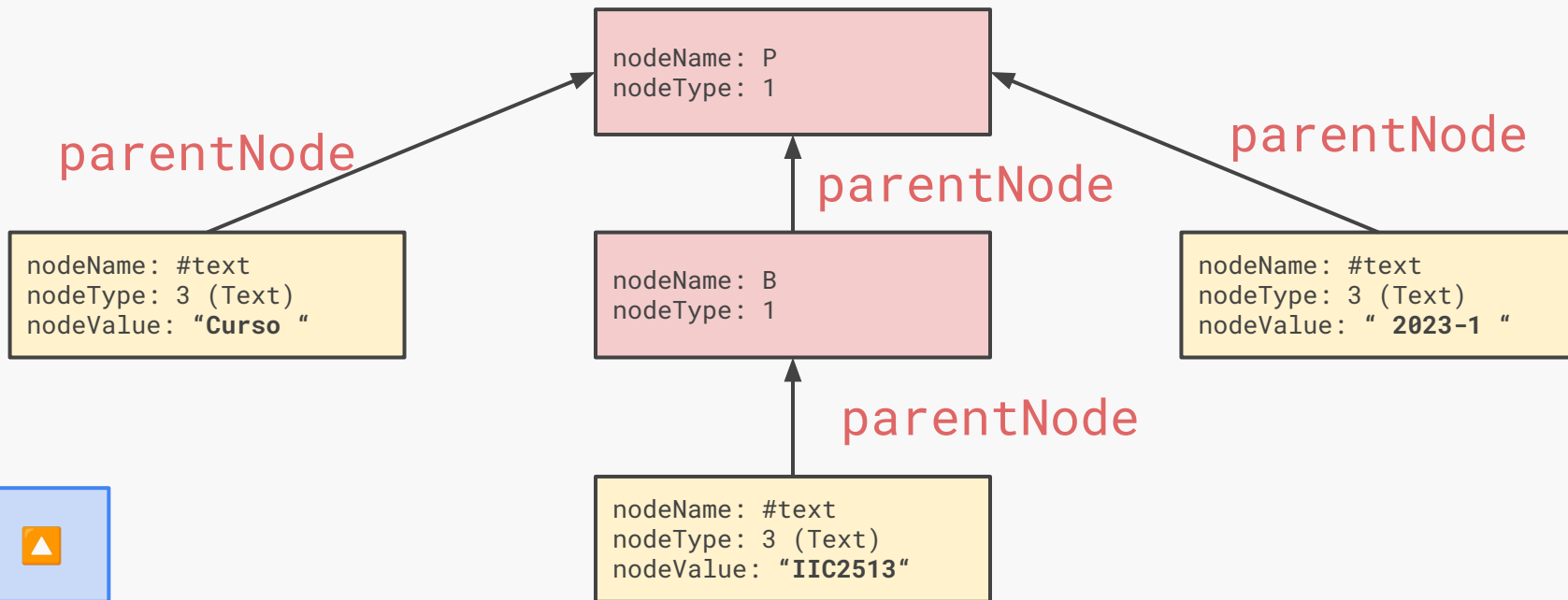
¿Vivo?
"Siempre reflejan el estado actual del documento y se actualizan cuando el documento cambia"

Algunas propiedades de nodos

- Identidad:
 - `nodeName` indica el tipo de elemento (P, DIV, etc.) o #identificador
- Jerarquía estructural:
 - `parentNode`
 - `nextSibling`, `previousSibling`
 - `nextElementSibling`, `previousElementSibling`
 - `firstChild`, `lastChild`
 - `firstElementChild`, `lastElementChild`
- Acceso a atributos de elementos:
 - `getAttribute`
 - `setAttribute`

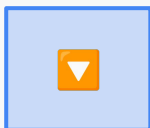
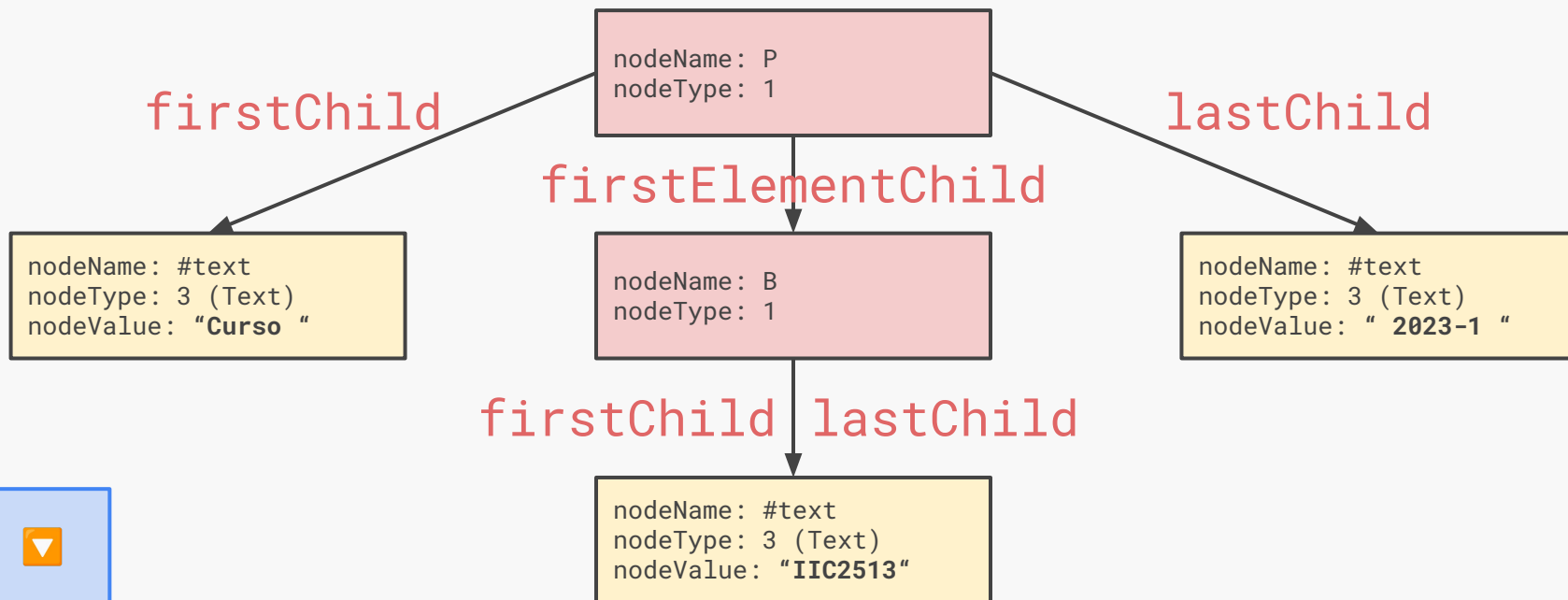
Algunas propiedades de nodos

`<p>Curso <b>IIC2513</b> 2025-1</p>`



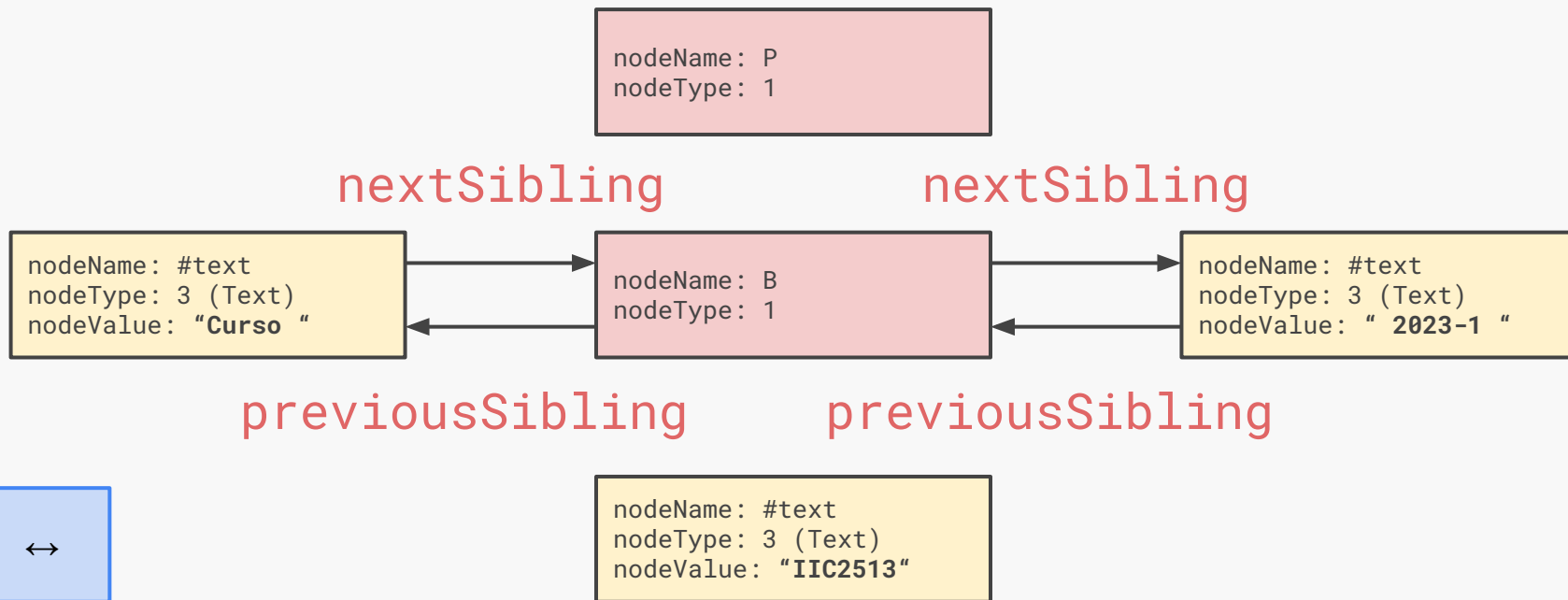
Algunas propiedades de nodos

`<p>Curso <b>IIC2513</b> 2025-1</p>`



Algunas propiedades de nodos

`<p>Curso <b>IIC2513</b> 2025-1</p>`



Algunos métodos adicionales

Relacionados con el contenido:

- `Node.textContent`: Retorna *string* del contenido de sus elementos
- `HTMLElement.innerText`: Entrega un *string* con el contenido “leíble”
- `Element.innerHTML`: Retorna HTML dentro de un elemento
- `Element.outerHTML`: Igual al anterior, pero incluye al elemento mismo
- `Element.getAttribute/setAttribute`: Atributos en un elemento

Relacionados con el el estilo:

- `element.classList.add("my-class")` agregar una clase
- `element.style.color` para cambiar el color
- `document.querySelector()` retorna el primer elemento que coincide con el selector (que debe ser un selector válido en CSS)

¿Cómo ejecutamos JS desde HTML?

El atributo `src` del `tag script` indica la ruta (URL o *filepath*) del archivo JS

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Ejemplo con JS</title>
6    </head>
7    <body>
8      <script src='programa.js' charset='utf-8'></script>
9    </body>
10 </html>
```

Ejemplo: control de DOM desde JS

Con la variable `document` accedemos a toda tipo información del HTML:

```
1 // Imprimimos el link donde está el documento
2 console.log(document.URL);
3
4 // Obtenemos la lista de hijos del documento
5 document.children; // [<html>]
6
7 // Obtenemos el elemento HTML con id="importante"
8 const elemento = document.getElementById("importante");
9
10 // Creamos un elemento cuyo tag es <p>
11 const parrafo = document.createElement("p");
```

Ejemplo: control de DOM desde JS

```
1 // Creamos un elemento cuyo tag es <p>
2 const parrafo = document.createElement("p");
3
4 // Creamos un elemento tipo texto
5 const texto = document.createTextNode("¡Soy un texto >.<!");
6
7 // Agregamos el texto al párrafo
8 parrafo.appendChild(texto);
```

Ejemplo: control de DOM desde JS

```
1 // Creamos un elemento con tag <img>
2 const elemento = document.createElement("img");
3
4 // Definimos la clase del elemento
5 elemento.className = "importante";
6
7 // Definimos un único id al elemento
8 elemento.id = "principal";
9
10 // Definimos un atributo al elemento
11 elemento.setAttribute("width", 200);
```

Agenda

Asincronía

Manejo de DOM

Manejo de elementos



Respondiendo a eventos

Eventos

Existen distintos tipos de eventos:

- Asociados al *mouse*: selección, *clicks*, movimiento, *hover*
- Asociados al teclado: presionado de teclas
- Asociados a la ventana: *resize* o cierre de ventana de navegador
- Asociados al foco: Foco ganado (focusin) o perdido (blur)
- Asociados a tiempo
- Cambio de valor en un *input*, *form* subido
- Y más:
 - Contenido de un elemento cambió
 - Página cargada
 - Imagen cargada
 - Excepciones

Manejo de eventos: *event listener*

Para definir un *event handler* se especifican tres cosas:

1. **Qué pasó:** el evento de interés ("**click**")
2. **Dónde pasó:** el elemento de interés (**btn**)
3. **Qué hacer:** código a ejecutar cuando el evento ocurre (*callback*)

```
1  const btn = document.querySelector("button");
2
3  function random(number) {
4      return Math.floor(Math.random() * (number + 1));
5  }
6  2                                1
7  btn.addEventListener("click", () => {
8      const rndCol = `rgb(${random(255)}, ${random(255)}, ${random(255)})`;
9      document.body.style.backgroundColor = rndCol;
10 });                                3
```

Maneras de definir un *event listener*

1. En el código HTML (atado a un elemento particular):

```
<button onclick="algunaFuncion();"></button>
```

2. Con JS usando el DOM (más mantenible y flexible):

```
btn.addEventListener("click", () => {
```

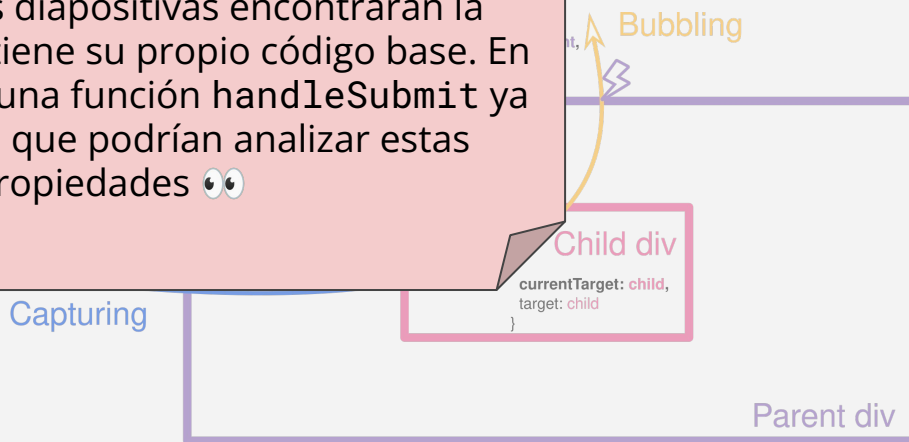

El objeto *event*

Los *event listener* reciben un objeto Event, con ciertas propiedades:

- **type**: nombre del evento
- **timeStamp**: momento en el que ocurrió el evento
- **currentTarget**: elemento que está escuchando el evento
- **target**: elemento que originó el evento

Dónde practicar 🧐

Al final de estas diapositivas encontrarán la **Actividad 2**, que tiene su propio código base. En ese código existe una función `handleSubmit` ya definida con la que podrían analizar estas propiedades 🧐



Agenda

Asincronía

Manejo de DOM



Actividad 2

¿Dónde está el código?

- Pueden encontrarlo en este Codepen. Si tienen una cuenta mejor aún, así pueden guardar su progreso, para lo que deben “Fork” (abajo a la derecha en la interfaz)

codepen.io/aaossa/pen/MWRmWXK

- También subí el código a este repositorio en la organización del curso en GitHub (ya saben qué hacer):

github.com/IIC2513/pizza-time

¿Cómo debería verse?

En la segunda parte deberán hacer código HTML dinámicamente:

Recibo de Compra
Fecha: 20/03/2025
Cliente: Juan Pérez

Ingrediente	Cantidad
Salsa	✓
Queso	✓
Pepperoni	5
Albahaca	3
Champiñones	4

Total: \$12.000

¡Gracias por su compra!



Recibo de Compra
Fecha: 20/03/2025
Cliente: Juan Pérez

Ingrediente	Cantidad
Salsa	✓
Queso	✓
Pepperoni	5
Albahaca	3
Champiñones	4

Total: \$12.000

¡Gracias por su compra!

Agenda

Asincronía

Manejo de DOM

Actividad 2



¿Qué se viene pronto?

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 2 (10/3 - 16/3)		Desde el navegador a mi sitio web		HTML y CSS			
Semana 3 (17/3 - 23/3)		JavaScript pt. 1 (Intro)		HOY	Ayudantía: JavaScript		
Semana 4 (24/3 - 30/3)		React		Exploración y consumo de APIs	Ayudantía: React		
Semana 5 (31/3 - 06/4)		Node.js		Desarrollando servidores	Ayudantía: Consumo de APIs		
Semana 6 (07/4 - 13/4)		Construcción de APIs		QA, CI & CD	Ayudantía: Construcción de APIs		

Próximas fechas importantes: evaluaciones

	Evaluaciones						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 2 (10/3 - 16/3)							
Semana 3 (17/3 - 23/3)				HOY			
Semana 4 (24/3 - 30/3)	Control 1				Inicio T1 React		
Semana 5 (31/3 - 06/4)					Fin T1 (React)		
Semana 6 (07/4 - 13/4)	Control 2				Inicio T2 API		

Clase 4

JavaScript parte 2

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl